Secure Coding Review

CodeAlpha Cyber Security Internship - Task 3

Application: Private Notes, a Python/Flask teaching application. **Review date:** 29 October 2026. **Assessment:** manual source review, local reproductions, static analysis, dependency auditing, and regression verification.

Executive summary

Six deliberately seeded weakness categories were reproduced in the teaching baseline and remediated in a separate corrected implementation. The verified effects include authentication bypass, cross-user note access and modification, stored script execution in another user's browser context, plaintext password storage, tokenless state changes, and forged sessions using a public signing key.

The final verification passed **58 Python tests and four browser checks**. Bandit reported four raw baseline warnings, one raw corrected-code warning about a published fictional seed password, and zero remaining warnings after narrowly documented triage. pip-audit reported no known vulnerabilities in nine pinned runtime packages or 45 pinned runtime/development packages at the recorded scan time.

The baseline was intentionally authored for this exercise. These are not discoveries in a third-party or production application. Severity describes the impact these patterns could have in a reachable application containing non-demo accounts and data. The delivered lab is bound to loopback, uses fictional data, and is not publicly hosted.

ID	Finding	Severity	Verification
SCR-01	SQL injection in login	High	Password bypass rejected after fix
SCR-02	Stored XSS, chained with SCR-03	High	Cross-user execution reproduced; corrected path blocked and escaped
SCR-03	Missing note ownership checks	High	Cross-user read/edit/delete rejected
SCR-04	Plaintext password storage	High	Stored hashes verified with legitimate login
SCR-05	Missing CSRF protection	Medium	Tokenless mutations rejected
SCR-06	Predictable session key and unsafe configuration	High	Forged cookie rejected; safer defaults verified

Address authentication and authorization boundaries first (SCR-01, SCR-03, SCR-06), then complete output-encoding, password-storage, and CSRF defenses before any real deployment. The corrected implementation applies all six groups of changes; the vulnerable snapshot remains available as

audit evidence.

Review scope and provenance

The application provides registration, login/logout, and private note CRUD through server-rendered forms. SQLite stores accounts and notes. Authentication state is held in a signed Flask cookie. Notes are plain text; HTML authoring is not an intended feature.

- Baseline commit: `7d0102c450733cde23cac9be27a1d804bb4a628a`.
- Reviewed corrected source commit: `f3c2e3428d767a4f2c2ea98fbc5139b01ee3f426`.
- The baseline application, templates, stylesheet, and runtime requirements are unchanged from the baseline commit. Shared plumbing gained only a documented scanner annotation for the published seed password.
- Source hashes, tool versions, exact commands, UTC timestamp, and exit codes are recorded in verification.json. Later documentation commits do not change the reviewed application bytes.
- The PDF formatter gained embedded fonts after evidence capture. That documentation-only tool update does not change the applications, tests, or recorded security results.
- Reviewed: both application packages, shared database/authentication helpers, templates, runner configuration, and pinned Python dependencies. Tests and browser probes verify specific security properties.
- Excluded: third-party services, real traffic, operating-system/container hardening, production hosting, an exhaustive dependency source review, and independent penetration testing.

The main trust boundaries are browser input entering SQL or templates, a signed cookie selecting an authenticated user, and a requested note identifier selecting a database object. A valid login must not authorize access to every note.

Method and evidence handling

Manual review traced registration, authentication, session setup, all note routes, rendering, and database operations. Bandit supplemented that review; pip-audit checked fully pinned package lists against the PyPI advisory service. Paired tests first confirm each baseline behavior and then assert its corrected counterpart. Separate browser checks confirm script execution/escaping and normal user flows.

All reproductions use temporary local databases with Alice and Bob as fictional users. The public seed password is a fixture. Script payloads only set a local JavaScript marker; no data is transmitted externally. Evidence contains no real passwords, signing keys, authentication tokens, session cookies, or application databases. Full command transcripts and tool JSON are preserved rather than replaced by screenshots alone.

SCR-01: SQL injection in login

Severity: High. **Classification:** CWE-89. **Prerequisite:** access to the login form and an existing account. No valid password is required.

Affected code: baseline `vulnerable/app.py`, line 47, at the baseline commit. The login statement interpolates both submitted fields into SQL.

```
user = get_db().execute(
    f"SELECT id FROM users WHERE username = '{username}' AND password = '{password}'"
).fetchone()
```

Reproduction and observed result

- In a fresh seeded baseline, submit username `alice' --` and an incorrect password to `/login`.
- The comment terminates the password condition. The server redirects with HTTP 302 and establishes Alice's session (`user_id` 1).
- The corrected application returns HTTP 401 and establishes no authenticated user for the same input, even when a valid CSRF token accompanies the request.

Impact: a caller can bypass authentication and obtain another account's note access. This finding does not rely on guessing the published demonstration password.

Remediation and verification

Corrected `secure/app.py`, lines 71–73, looks up the submitted username using `WHERE username = ?` and verifies a stored password hash. Input remains data rather than part of SQL syntax. Parameterization is used throughout the corrected application; restrictive username rules are additional validation, not the injection defense.

Evidence: `tests/test_findings.py::test_scr01_sql_injection_login`, recorded in pytest.txt and pytest.xml. Bandit B608 flags the baseline interpolation with medium severity/confidence; the verified authentication bypass justifies this report's High rating. Normal login, incorrect passwords, nonexistent users, and apostrophes are also tested.

SCR-02: Stored XSS through trusted HTML rendering

Severity: High when chained with SCR-03. **Classification:** CWE-79. **Prerequisite:** attacker-controlled text must be viewed by another user. In this lab, SCR-03 supplies that cross-user write path; ordinary access to only one's own notes would demonstrate self-execution rather than a cross-user compromise.

Affected code: baseline `vulnerable/app.py`, line 87, wraps stored note text in `Markup`. The shared template then renders it as trusted HTML.

```
return render_template(
    "note.html", note=note, note_content=Markup(note["body"])
)
```

Reproduction and observed result

- Sign in as Alice and use the missing ownership check to edit Bob's note at `/notes/2/edit`.
- Save the harmless payload below, sign out, sign in as Bob, and open `/notes/2`.

```
<script>window.__xssProbe="executed"</script>
```

- Real Chromium execution sets `window.__xssProbe` to `executed` in Bob's page. The stored text is interpreted as code.
- The corrected app returns HTTP 404 when Alice requests Bob's edit form. For a note Alice is allowed to create, the same payload appears literally as text and the marker remains unset.

Impact: malicious stored HTML can execute with the application's origin when a victim views the planted note. The baseline also omits HttpOnly; this exercise deliberately does not access or export cookies. The report does not infer a cross-user attack solely from a payload running in its author's own note.

Remediation and verification

Corrected `secure/app.py`, line 112, passes ordinary text to Jinja's automatic escaping. It does not use `Markup` or a `safe` filter for note content. Ownership checks remove the cross-user write path. A restrictive CSP with `script-src 'none'` adds protection; escaping is independently asserted in the response and browser-visible text.

Evidence: `test_scr02_stored_xss` in the paired Python suite; the two stored-XSS browser checks in `tests/browser.test.cjs`; and browser-observations.json. Bandit B704 also flags the unsafe baseline `Markup` call.

SCR-03: Missing object-level authorization

Severity: High. **Classification:** CWE-639. **Prerequisite:** a signed-in user and another user's note identifier. Sequential identifiers are easy to discover, but the defect is missing authorization rather than the identifier format.

Affected code: baseline `vulnerable/app.py`, lines 75–80, retrieves a note by ID alone. The view, edit, and delete routes reuse this helper.

```
note = get_db().execute(
    "SELECT * FROM notes WHERE id = ?", (note_id,)
).fetchone()
```

Reproduction and observed result

- Sign in as Alice in a fresh isolated fixture. Bob owns note 2.
- `GET /notes/2` and `GET /notes/2/edit` return HTTP 200.
- A valid edit submission changes Bob's content; a delete submission removes Bob's note. Both return HTTP 302.
- The corrected application returns HTTP 404 for every operation. Tests supply valid CSRF tokens where required, proving that ownership validation is the blocking control. Bob's data remains unchanged.

Impact: an authenticated attacker can read, alter, or delete another user's private information. This also enables the stored-XSS chain documented in SCR-02.

Remediation and verification

Corrected `secure/app.py`, lines 101–106, selects notes using both ID and authenticated owner ID. Updates and deletes also include the owner condition. The server derives identity from its verified session; it never trusts an owner ID supplied by a form. Missing and non-owned notes share a 404 response.

Evidence: `test_scr03_cross_user_access` covers read, edit-form access, update, and delete in both versions. The browser test separately verifies Alice's baseline access to Bob's edit form and the corrected 404. The tested Bandit configuration did not identify this business-logic defect; manual review was essential.

SCR-04: Plaintext password storage

Severity: High, conditional on disclosure of the account database or backups. **Classification:** CWE-256. This finding does not itself demonstrate a database-download endpoint or remote filesystem access.

Affected code: baseline `vulnerable/app.py`, line 35, inserts the submitted password directly; line 23 also leaves seeded passwords unchanged. The login query compares plaintext values.

Reproduction and observed result

- Register a fictional new user through `/register`.
- Inspect that account in the isolated SQLite fixture. The stored password equals the submitted password byte for byte.
- In the corrected app, the stored value differs from the input and is a Werkzeug scrypt hash. `check_password_hash` verifies the correct password; normal sign-in still succeeds.

Impact: a database or backup leak immediately exposes usable passwords, including possible reuse on other services. Password strength rules cannot compensate for reversible or plaintext storage.

Remediation and verification

Corrected `secure/app.py`, lines 58–59, hashes new passwords with `generate_password_hash`; lines 71–72 verify using `check_password_hash`. The seed path uses the same hash helper. Werkzeug generates salts and includes the algorithm/cost in the stored representation. Production policy should assess hashing cost against current guidance and available resources.

The two versions use separate databases. This project does not silently import plaintext accounts into the corrected app. A real migration would need a planned reset or controlled rehash process and secure treatment of old backups.

Evidence: `test_scr04_password_storage` checks registration, stored representation, hash verification, and login. `test_database_setup_is_repeatable_and_hashes_seed_passwords` verifies seeded accounts. Bandit did not infer the plaintext-storage defect from this data flow.

SCR-05: Missing CSRF protection

Severity: Medium. **Classification:** CWE-352. **Prerequisite:** a forged form submission must arrive with the victim's authentication cookie. Browser cookie policies affect exploitability; a request without a token being accepted is not by itself proof that every unrelated website can send an authenticated request.

Affected code: the baseline factory has no CSRF protection and no form tokens. State-changing routes include registration/login, note creation/edit/deletion, and logout.

Reproduction and observed result

- Authenticate in an isolated baseline fixture, then submit an otherwise valid note-create, edit, delete, or logout POST without a CSRF token.
- Each baseline request is accepted with HTTP 302.
- The corrected app returns HTTP 400, leaves notes unchanged, and preserves the authenticated session for the rejected logout.
- Separate tests reject missing tokens on authentication forms, malformed tokens, and tokens issued to another browser session. Normal forms with valid tokens continue to work.

Impact: where the browser sends the session cookie, a forged request can change data or log out the victim. Default SameSite handling can reduce some cross-site paths, while same-site but different-origin contexts may remain relevant. Ownership checks alone do not address a forged request made as the owner.

Remediation and verification

Corrected `secure/app.py`, line 33, registers `CSRFProtect` globally. Every POST form includes its session-bound token. Token lifetime is one hour. Logout is POST-only. SameSite=Lax provides an additional cookie control; it does not replace token verification.

Evidence: paired `test_scr05_missing_csrf` cases; corrected authentication-form and cross-session-token tests in `tests/test_secure_app.py`; and pytest.xml. The tested Bandit configuration did not flag the missing framework-wide CSRF policy.

SCR-06: Predictable session signing key and unsafe configuration

Severity: High because the known signing key permits impersonation. **Primary classification:** CWE-321. Related hardening issues are CWE-489 (active debug code/configuration) and CWE-1004 (missing HttpOnly cookie attribute).

Affected code: baseline `vulnerable/app.py`, lines 13–18, embeds a public signing key, enables debugging, disables HttpOnly, and does not specify SameSite. The runner consumes the application's debug setting.

Reproduction and observed result

- Using the published baseline key and Flask's signing serializer, construct a cookie containing `user_id` 2.
- Send it as the baseline session cookie without signing in. `GET /notes/2` returns HTTP 200 as Bob.
- Send the same signed value under the corrected cookie name. It fails signature verification; the request redirects to login with HTTP 302.
- Tests confirm debug is disabled and corrected login cookies contain HttpOnly and SameSite=Lax. Missing, empty, and short environment keys prevent corrected startup.

Impact: knowledge of the signing key permits forgery of account identity. Active debug configuration can disclose diagnostic information if exposed. The lab does not demonstrate arbitrary debugger execution and makes no claim that a debugger PIN was bypassed.

Remediation and verification

Corrected `secure/app.py`, lines 15–31, requires an environment-provided signing key of at least 32 characters, with no built-in fallback. The README generates it with Python's cryptographic `secrets` module. Debug is disabled; cookies are HttpOnly and SameSite=Lax. Login clears pre-authentication state; logout clears the browser session. Sessions have a 30-minute rolling lifetime.

Cookie Secure remains disabled because this lab serves local HTTP. A real HTTPS deployment must enable Secure and configure TLS and a production server. Cookies are signed, not encrypted. Logout is not centralized revocation: a previously stolen valid signed cookie can remain usable until its signature age expires or the key changes.

Evidence: the paired session-forgery and cookie-flag tests; startup-secret and session-lifecycle tests; and the verification manifest. Bandit B106 flags the public baseline key. The config-driven debug path was found manually rather than by the recorded scanner.

Verification results and scanner triage

All results below were captured from corrected source commit `f3c2e3428d767a4f2c2ea98fbc5139b01ee3f426` on 29 October 2026 using Python 3.12.14, Flask 3.1.3, Flask-WTF 1.3.0, Werkzeug 3.1.9, pytest 9.2.0, Bandit 1.9.4, pip-audit 2.10.1, Playwright 1.62.1, and Chromium. ReportLab 5.1.0 renders this Markdown as PDF.

Check	Observed result	Evidence
Paired security and functional tests	58 passed; no failures	`evidence/pytest.txt`, `pytest.xml`
Real-browser verification	4 passed; XSS chain, corrected CRUD, keyboard and layouts	`evidence/browser.txt`, `browser-observations.json`
Raw baseline Bandit	4 warnings: B105, B106, B608, B704	`evidence/baseline-bandit.json`
Raw corrected Bandit	1 warning: public seed fixture B105	`evidence/secure-bandit-raw.json`
Triaged corrected Bandit	0 remaining warnings; one explicit fixture suppression	`evidence/secure-bandit.json`
Runtime dependency audit	9 packages; no known vulnerabilities reported	`evidence/runtime-dependencies.json`
Runtime/development dependency audit	45 packages; no known vulnerabilities reported	`evidence/development-dependencies.json`

The Bandit B105 warning at `labcore.py`, line 11, is a **reviewed fixture**, not a confidential credential: both demo account names and their password are intentionally published. Seeding is opt-in, uses only local fictional accounts, and hashes their passwords in the corrected database. One `nosec B105` annotation suppresses exactly that warning in the gating scan; `--ignore-nosec` raw reports preserve it for review. No application signing key is suppressed. Default demo accounts must not be carried into a real deployment.

The raw baseline scan was run against the preserved baseline application plus current shared plumbing. The only plumbing change from the original commit is the documented fixture annotation; raw scanning ignores it. B608 and B704 are medium-severity static warnings; the manually verified login bypass and cross-user execution determine this report's ratings. Scanner counts are not the number of verified findings.

Dependency results apply to the pinned package lists and advisory service at the recorded timestamp. No advisories were ignored. The two dependency checks use fully pinned requirements with dependency resolution disabled; the manifests include resolved transitive packages. They do not audit Node/browser packages or establish that dependencies have no undiscovered defects. Playwright is test tooling, not an application runtime dependency.

Reproduce the verification

After completing the Python and browser setup in the README:

```
python tools/run_review.py
```

The runner records expected nonzero exits for the raw baseline/fixture warnings and requires successful corrected-code scans, tests, and dependency audits. Unexpected failures or timeouts produce a nonzero overall result. It uses temporary databases for tests and records SHA-256 hashes of the reviewed source. Browser servers are stopped automatically.

To inspect a single category:

```
python -m pytest tests/test_findings.py -k scr01 -v
python -m pytest tests/test_findings.py -k scr03 -v
```

GitHub Actions runs the same categories on pushes and pull requests. A workflow file's presence is not evidence of a successful hosted run; use the repository's Actions result for that status.

Remaining limitations and secure coding recommendations

- Keep the application local. Deployment would require TLS, Secure cookies, a production WSGI server, proxy/host configuration, operational secrets management, backups, monitoring, and incident procedures.
- Add rate limits, breached-password screening, account recovery, MFA where appropriate, and explicit account-enumeration/timing policy before treating the application as a real authentication service.
- Assess scrypt cost parameters against current password-storage policy. Never log submitted passwords or carry demo credentials into real accounts.
- Consider server-side session revocation and an absolute lifetime if the threat model requires them; client-side signed cookies alone do not provide these controls.
- Apply authorization to every object access, including future exports, search endpoints, and APIs. Parameterized queries do not enforce ownership.
- Preserve output encoding when adding rendering features. Rich text would need a separate allowlist-sanitization design; plain text is the current interface contract.
- Keep dependencies pinned, rescan advisories, and review updates. The test suite covers documented cases, not every input or deployment configuration.

The initial baseline remains intentionally vulnerable. The corrected implementation addresses the six demonstrated categories under the local lab's assumptions. No production security certification or exhaustive audit is claimed.

References

- OWASP Code Review Guide.
- OWASP Cheat Sheet Series: SQL injection, XSS, IDOR, password storage, CSRF, and session management.
- Flask security considerations.

- CWE-89, CWE-79, CWE-639, CWE-256, CWE-352, and CWE-321.
- Full source and tool references are listed in SOURCES.md.